

LUDORA

Documentação técnica do projeto

Sistema de gestão de clubes de futsal/futebol — App, Web e Desktop

Este documento registra a arquitetura, as decisões técnicas e o estado atual do projeto Ludora, de forma que qualquer pessoa do grupo — ou uma nova conversa com o Claude — consiga retomar o contexto completo sem depender de memória do que já foi discutido.

1. Contexto e histórico do projeto

O Ludora nasceu como projeto de extensão universitária, com dois requisitos obrigatórios: (1) ser um aplicativo e (2) incluir um sistema de machine learning. A partir disso foi desenvolvido pensando no CFA Ocian Praia Clube (Praia Grande/SP), focado em futsal — um sistema para digitalizar a súmula de partida (antes preenchida em papel), permitir acompanhamento em tempo real dos jogos e usar ML para gerar um perfil individual de desempenho de cada atleta (o "Scout").

No semestre seguinte, um novo projeto de extensão surgiu, com requisitos diferentes: o sistema precisa existir em **três plataformas distintas** — app, web e desktop — e cada uma precisa ter uma função genuinamente diferente das outras (não pode ser a mesma tela copiada três vezes). A parte de ML deixou de ser obrigatória. Decidiu-se manter e evoluir o Ludora em vez de recomençar, aproveitando a base já validada com o Ocian, mas agora com ambição de virar um produto real, vendável para múltiplos clubes (modelo SaaS multi-clube), não mais hardcoded para um único cliente.

2. Divisão dos três sistemas

A divisão foi pensada por **momento de uso e persona**, não por conveniência técnica — cada sistema tem um público e um contexto de uso diferente:

Sistema	Função	Quem usa
App (React Native / Expo)	Dia de jogo: iniciar partida, registrar gol/cartão/falta em tempo real, acompanhar placar ao vivo.	Mesário, pais, torcida
Web (React)	Vitrine pública: institucional, cadastro de novos clubes, classificação e calendário públicos.	Visitante casual, novos clubes-cliente
Desktop (Electron + React)	Gestão do clube: cadastro de atletas, montagem de elenco/campeonato, importação da tabela da federação.	Administradores/gestores do clube

O backend (API Node + serviço de ML em Python) é **único e compartilhado** pelos três — não existe "banco do app" e "banco do web" separados. A separação em três sistemas é de interface e de quais rotas cada um consome, não do modelo de dados.

Stack escolhida para o desktop: **Electron + React**, reaproveitando componentes de UI e chamadas de API do sistema web (ambos React), evitando duplicar trabalho. Alternativa considerada e descartada por ora: Tauri (binário mais leve, mas exige backend em Rust).

3. Estrutura do repositório (monorepo)

O repositório foi reorganizado de uma estrutura solta (backend-ml, backend-node, cfaOcian como pastas soltas) para um **monorepo** gerenciado por **pnpm workspaces**, preparado para receber web e desktop sem precisar reestruturar tudo de novo.

```
ludora/
├── apps/
│   ├── mobile/ (era cfa0cian – React Native / Expo)
│   ├── web/ (placeholder – a criar)
│   └── desktop/ (placeholder – a criar)
├── services/
│   ├── api/ (era backend-node – Express + Prisma)
│   └── ml/ (era backend-ml – FastAPI / Python)
├── packages/
│   ├── shared-types/ (tipos TS compartilhados entre apps)
│   └── ui/ (componentes React compartilhados: web + desktop)
├── docs/ (documentação, diagramas, decisões)
├── pnpm-workspace.yaml
├── .npmrc (shamefully-hoist=true)
├── iniciar-os-terminais.bat
└── package.json
```

docs/ — documentação que não é código: diagramas, decisões arquiteturais (ADRs), guias de setup.

packages/ — código consumido por mais de um app; hoje vazios, prontos para receber tipos e componentes assim que web/desktop existirem.

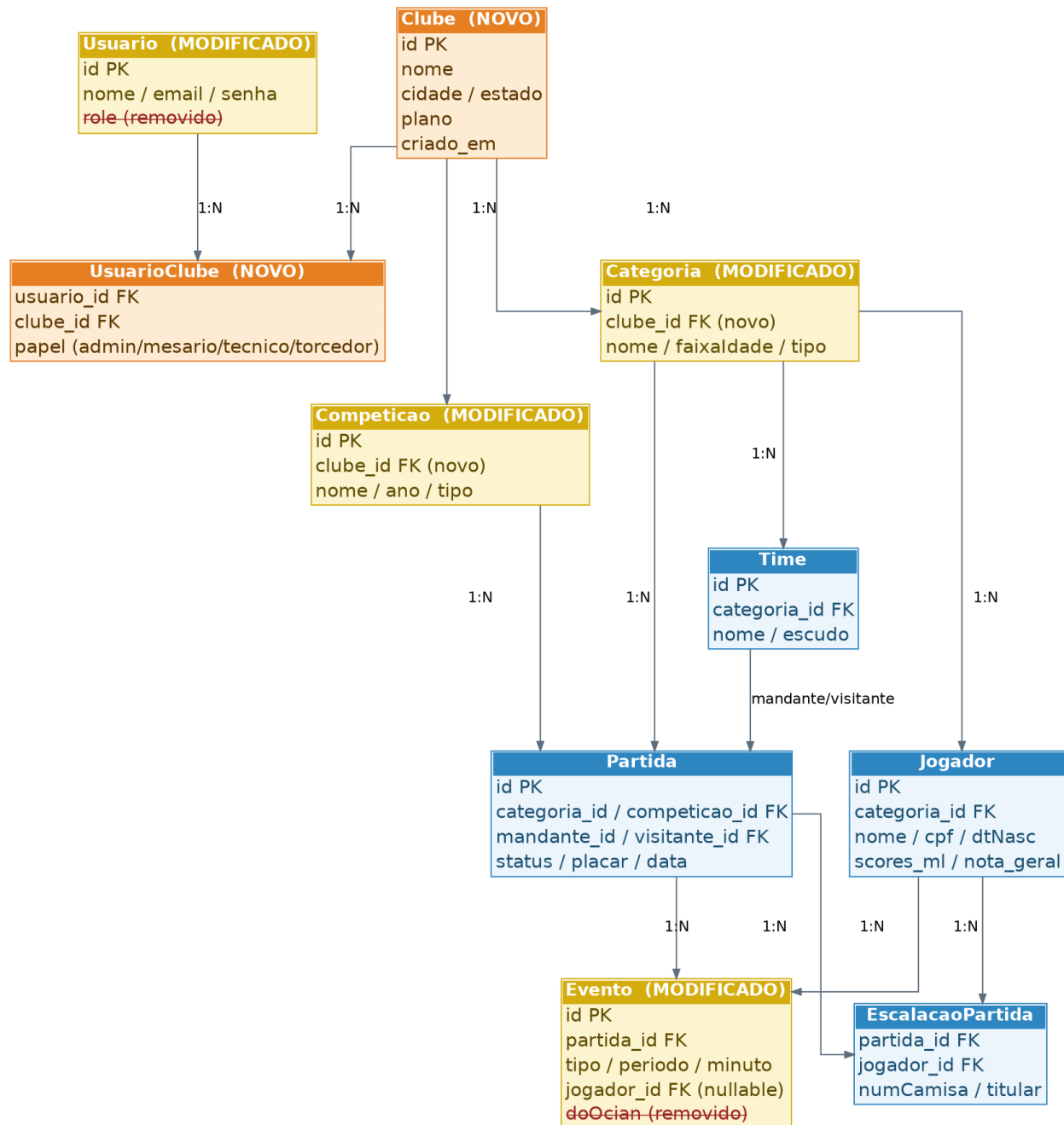
3.1 Migração realizada

- Pastas movidas com `git mv` (preserva histórico do Git, ao contrário de um `mv` comum).
- Pacotes renomeados para o escopo do workspace: `cfaocian` → `@ludora/mobile`, `backend-node` → `@ludora/api`.
- `.npmrc` com `shamefully-hoist=true` na raiz — necessário porque o Metro Bundler (Expo/React Native) não resolve bem a estrutura de symlinks padrão do pnpm em monorepos.
- `pnpm approve-builds` executado para liberar scripts de build de pacotes nativos (Prisma Client, `bcrypt`, `esbuild`), bloqueados por padrão por segurança.
- `iniciar-os-terminais.bat` atualizado com os novos caminhos (`apps\mobile`, `services\api`, `services\ml`) e mantido versionado no Git (não é segredo, e o time inteiro precisa dele).

Cuidado registrado: evitar rodar o projeto dentro de uma pasta sincronizada por OneDrive/Google Drive — o processo de sync trava handles de arquivos binários (ex: engine do Prisma, `bcrypt.node`), causando erro de permissão ao tentar apagar/reinstalar `node_modules`.

4. Modelo de dados — situação atual e plano de migração

O schema atual (Prisma + PostgreSQL) foi construído para um único clube (Ocian), sem conceito de "tenant". A tabela Evento tem, inclusive, um campo booleano literal doOcian. Para o objetivo de multi-clube (SaaS), é necessário migrar para um modelo multi-tenant. O diagrama abaixo mostra o modelo atual e o que precisa mudar.



Legenda: laranja = entidade nova · amarelo = entidade existente que precisa mudar · azul = entidade existente que permanece igual.

4.1 O que precisa ser criado

- Clube** — nova tabela raiz (id, nome, cidade/estado, plano, criado_em). É o "dono" de tudo: categorias, competições, atletas (indiretamente via categoria).

- **UsuarioClube** — tabela pivot (`usuario_id`, `clube_id`, `papel`). Substitui o campo `Usuario.rol` atual (enum global ADMIN/USER). Necessária porque, com múltiplos clubes, a mesma pessoa pode ter papéis diferentes em clubes diferentes (ex: admin no Ocian, torcedor em outro clube). Papéis previstos: admin, mesário, técnico, torcedor.

4.2 O que precisa ser alterado

- **Categoria** — adicionar `clube_id` (FK). A constraint `@unique` no campo `nome` hoje é global ("sub-7" só pode existir uma vez no sistema inteiro) — precisa virar `@@unique([nome, clube_id])`, já que dois clubes diferentes terão, cada um, seu próprio "sub-7".
- **Competicao** — adicionar `clube_id` (FK), mesma lógica.
- **Evento** — remover o campo `doOcian` (boolean hardcoded). A informação de "a favor de qual time" já é dedutível via `jogador_id` → `categoria` → `clube`, ou pelo lado (mandante/visitante) da partida.
- **Usuario** — remover o campo `rol` (enum simples), substituído pela relação com `UsuarioClube`.

4.3 O que permanece igual

`Time`, `Jogador`, `Partida` e `EscalacaoPartida` não precisam de alteração estrutural — todos já pendem, direta ou indiretamente, de `Categoria`, então herdam o escopo de clube automaticamente assim que ela ganhar `clube_id`. É uma boa notícia: o núcleo mais complexo do domínio (partida, evento, escalação) não precisa ser refeito.

4.4 Impacto fora do schema

- **Rotas da API** — hoje o `server.ts` (aprox. 35 rotas, todas no mesmo arquivo) faz consultas sem nenhum filtro de clube. Cada rota vai precisar de um middleware que resolve o clube ativo (via JWT ou header) e filtra os dados por ele.
- **JWT** — hoje carrega apenas `{id, rol}`. Precisa passar a carregar os vínculos do usuário com seus clubes e papéis.
- **App (auth.ts)** — a função `isAdmin()` hoje é global (lida direto do `SecureStore`); precisa considerar qual clube está selecionado no momento.

Essa migração pode (e deve) ser feita de forma incremental: primeiro a migration do Prisma, depois as rotas sendo ajustadas uma a uma — não precisa ser um refactor único e arriscado.

5. Stack técnica atual (levantada do código)

Camada	Tecnologia	Observações
App mobile	React Native + Expo Router	Autenticação via SecureStore; roteamento em app/(auth) e app/(tabs)
API principal	Express + Prisma + PostgreSQL	JWT (jsonwebtoken + bcrypt), Socket.io já configurado para tempo real, cron a cada 30 min sincronizando classificação via scraping da federação
Serviço de ML	Python (FastAPI / Uvicorn)	Serviço separado, chamado pela API via PYTHON_AI_URL; grava resultado em Jogador.scores_ml (JSON)
Gerenciador de pacotes	pnpm (workspaces)	shamefully-hoist=true necessário para compatibilidade com Metro/Expo

6. Setup local — checklist rápido

- Repositório fora de pastas sincronizadas por OneDrive/Google Drive.
- `npm install -g pnpm` (uma vez, global).
- `pnpm install` na raiz do monorepo.
- `pnpm approve-builds` (libera build scripts do Prisma, bcrypt, esbuild).
- Dentro de `services/api`: `npx prisma generate`.
- Conferir `BASE_URL` em `apps/mobile/src/services/api.ts` — precisa ser o IP local (LAN) da máquina rodando a API, não `localhost`, para o Expo Go (rodando no celular) conseguir alcançar o backend. Celular e computador precisam estar na mesma rede Wi-Fi.
- Rodar `iniciar-os-terminais.bat` para subir mobile, api e ml de uma vez.

7. Próximos passos

- Escrever e aplicar a migration do Prisma com as mudanças da seção 4 (Clube, UsuarioClube, clube_id em Categoria/Competicao, remoção de doOcian e role).
- Ajustar as rotas da API para filtrar por clube.
- Iniciar `apps/web` (landing institucional + página pública por clube).
- Iniciar `apps/desktop` (Electron + React, reaproveitando `packages/ui`).
- Popular `packages/shared-types` com os tipos derivados do schema atualizado.
- Decidir o destino da parte de ML: manter simplificada, repensar como diferencial, ou postergar (discussão em aberto).